Proyecto #1

Programación concurrente

Nombre del estudiante:
Joed Francisco Sosa Castellanos

Número de cuenta:
12211012

Sede de estudio:
Campus Tegucigalpa

Docente:
Ing.Roman Pineda

Sección:
CCC415

Fecha de entrega:
15 de agosto de 2026

Índice

Introducción....4
Objetivos....4
Entorno de trabajo....4
Paso 1: Bubble Sort secuencial con contador de intercambios....7
Modificación realizada....7
Ejecución con el ejemplo del enunciado....8
Pruebas con 10, 100, 1,000 y 10,000 elementos....9
Resultados con puntos intermedios....9
Interpretación de los resultados....11
Paso 2: División del arreglo en dos mitades y mezcla (merge)....12
Descripción del algoritmo....12
Ejecución con el ejemplo del enunciado....13
Pruebas con 10, 100, 1,000 y 10,000 elementos....14
Comparación con el Paso 1....14
Conclusiones del Paso 2....15
Paso 3: Ordenamiento de las dos mitades en hilos separados....17
Conceptos de hilos en Java....17
Implementación....17
Ejecución....19
Comparación de las tres versiones....20
Uso de los núcleos durante la ejecución....22
Simulación de un CPU single-core y dual-core....23
Conclusiones del Paso 3....26
Demostración de una condición de carrera....27
Comentarios de implementación y dificultades encontradas....29
Conclusiones generales....30
Referencias....31
Anexo: Código fuente....32
bubbleSort.java (Paso 1)....32
bubbleSortDividido.java (Paso 2)....33

bubbleSortHilos.java (Paso 3).....35
Comparacion.java (experimento comparativo).....37

Introducción

La programación concurrente reúne las técnicas que permiten que varias partes de un programa avancen al mismo tiempo. Hoy prácticamente cualquier computadora tiene un procesador con varios núcleos, de modo que un programa que se ejecuta en un único hilo aprovecha solo una fracción del CPU disponible. Para el sistema operativo este tema está ligado a la gestión de procesos e hilos, ya que es el planificador (scheduler) quien decide en qué núcleo y en qué momento se ejecuta cada hilo (Silberschatz et al., 2018).

En este proyecto se parte de un algoritmo sencillo, el Bubble Sort, y se transforma en tres pasos para observar con datos medidos qué ocurre con el número de intercambios y con el tiempo de ejecución (Pineda, 2026). En el primer paso se usa el Bubble Sort clásico modificado para contar los intercambios. En el segundo, el arreglo se divide en dos mitades que se ordenan por separado y luego se mezclan (merge), todavía de forma secuencial. En el tercero, cada mitad se ordena en un hilo (thread) de Java distinto y, cuando ambos terminan, se hace un único merge.

Objetivos

El objetivo general es dar los primeros pasos en la programación concurrente con los hilos de Java y medir su efecto real sobre el rendimiento. De forma específica, se busca medir cuántos intercambios realiza el Bubble Sort para 10, 100, 1,000 y 10,000 elementos y explicar ese comportamiento con la teoría; determinar si dividir el problema en dos mitades mejora el rendimiento y por qué; implementar la versión con hilos usando Thread, start y join y compararla con las versiones secuenciales; y analizar cómo se comportaría la versión con hilos en un CPU de un solo núcleo o de dos núcleos, simulándolo en la práctica al limitar los procesadores que puede usar el programa.

Entorno de trabajo

Todas las pruebas se realizaron en una laptop con Windows 11, usando Visual Studio Code como editor y la terminal integrada para compilar y ejecutar. Las características del equipo se resumen en la Tabla 1.

Tabla 1. *Entorno en el que se obtuvieron los resultados.*

Elemento	Valor
Sistema operativo	Windows 11 de 64 bits
Procesador	AMD Ryzen 5 7520U, 2.80 GHz base (hasta 4.1 GHz)
Núcleos	4 núcleos físicos, 8 procesadores lógicos (SMT)
Memoria caché	L1: 256 KB, L2: 2 MB, L3: 4 MB
Memoria RAM	7.2 GB
Java	JDK 27 (javac 27)
Editor y terminal	Visual Studio Code con Extension Pack for Java, PowerShell y CMD
Datos de prueba	Enteros aleatorios entre 0 y 99,999 con semilla fija (12345 + N)
Medición de tiempo	System.nanoTime(), mediana de 5 repeticiones con calentamiento previo de la JVM

El proyecto está formado por cuatro programas sin paquete (package), guardados en la misma carpeta: `bubbleSort.java` para el Paso 1, `bubbleSortDividido.java` para el Paso 2, `bubbleSortHilos.java` para el Paso 3 y `Comparacion.java`, que ejecuta las tres versiones con exactamente los mismos datos, repite cada medición cinco veces y guarda los resultados en un archivo CSV que se puede abrir en Excel.

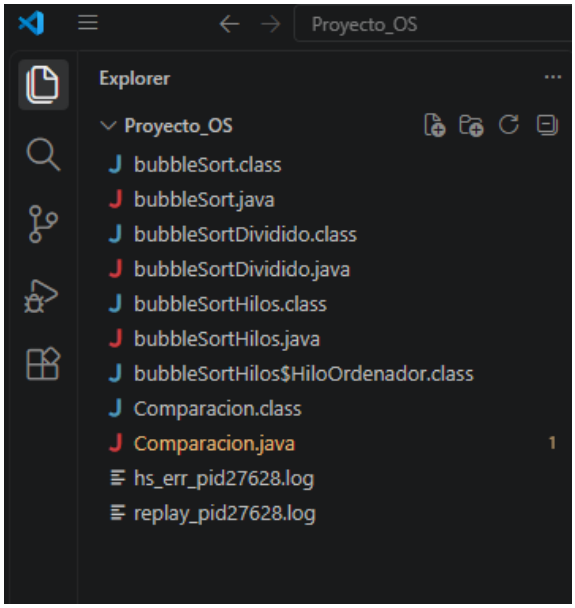


Figura 1. *Carpeta del proyecto en Visual Studio Code con los archivos fuente y compilados.*

Para compilar se abrió la terminal integrada de Visual Studio Code en la carpeta del proyecto y se ejecutó `javac *.java`. Cuando el comando no muestra ningún mensaje significa que

los cuatro programas compilaron sin errores. Después, cada programa se ejecuta con `java` seguido del nombre de la clase y, si se desea, de los tamaños de arreglo a probar.

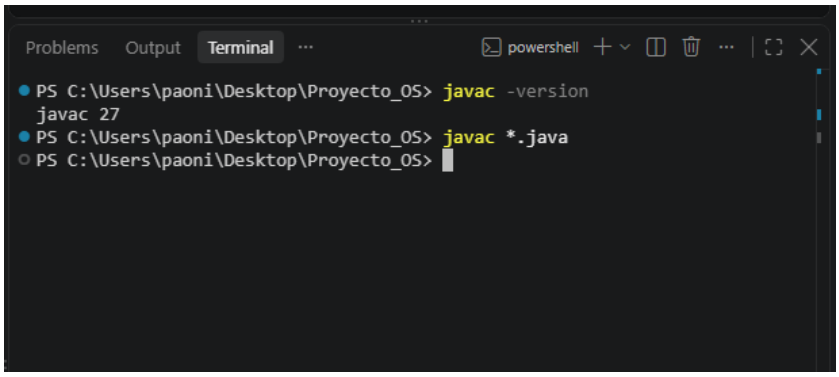


Figura 2. *Verificación de la versión del compilador (javac 27) y compilación de los cuatro programas.*

Como los datos se generan con una semilla fija, los tres programas ordenan exactamente los mismos números para cada tamaño N. Por eso el número de intercambios se puede comparar directamente entre pasos y es idéntico en cualquier computadora, mientras que los tiempos dependen del equipo y varían un poco entre ejecuciones.

Paso 1: Bubble Sort secuencial con contador de intercambios

Modificación realizada

Se transcribió el programa del enunciado en el archivo `bubbleSort.java`. La modificación pedida consiste en contar los intercambios, así que dentro del método `bubble_srt` se agregó la variable `intercambios`, que aumenta en uno cada vez que se ejecuta el bloque de intercambio (swap), y el método ahora devuelve ese total para que el `main` lo imprima. Se usó el tipo `long` en lugar de `int` porque con arreglos grandes el número de intercambios puede superar el valor máximo de un `int`, que es 2,147,483,647.

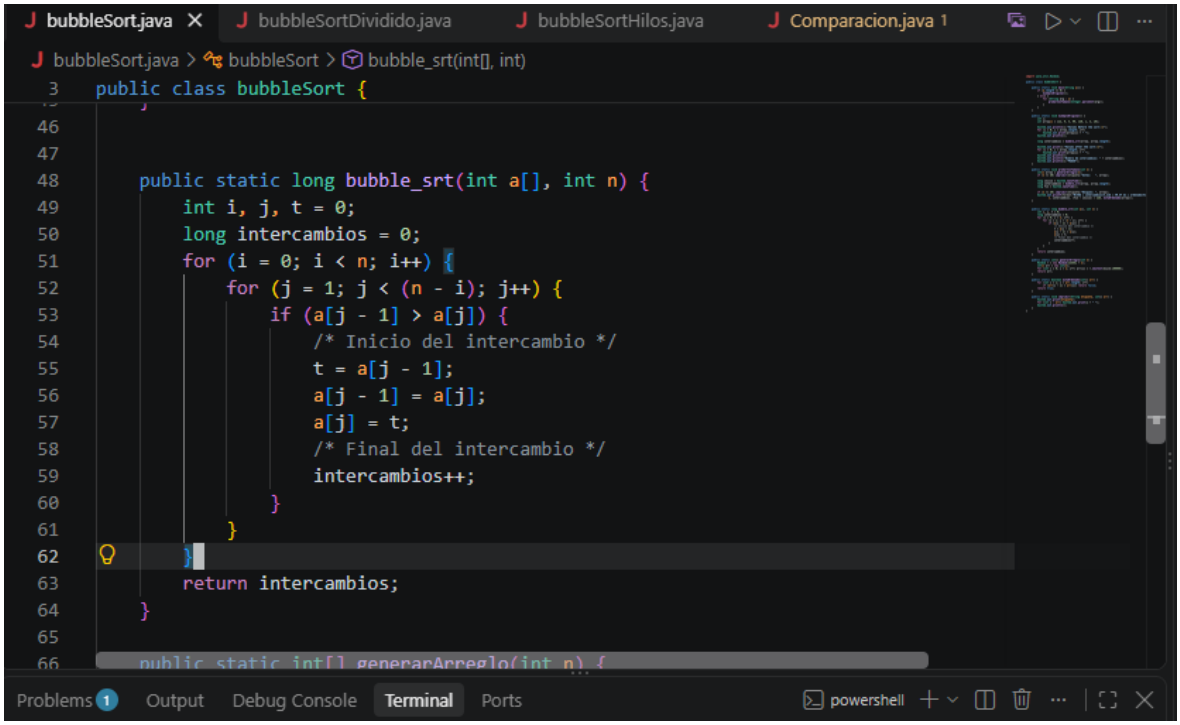


Figura 3. *Método bubble_srt con el contador de intercambios agregado.*

Además, el `main` acepta argumentos. Si se ejecuta sin argumentos corre el ejemplo original de ocho números; si se le pasan tamaños, genera arreglos aleatorios de esos tamaños, los ordena, verifica que quedaron ordenados y reporta intercambios y tiempo.

Ejecución con el ejemplo del enunciado

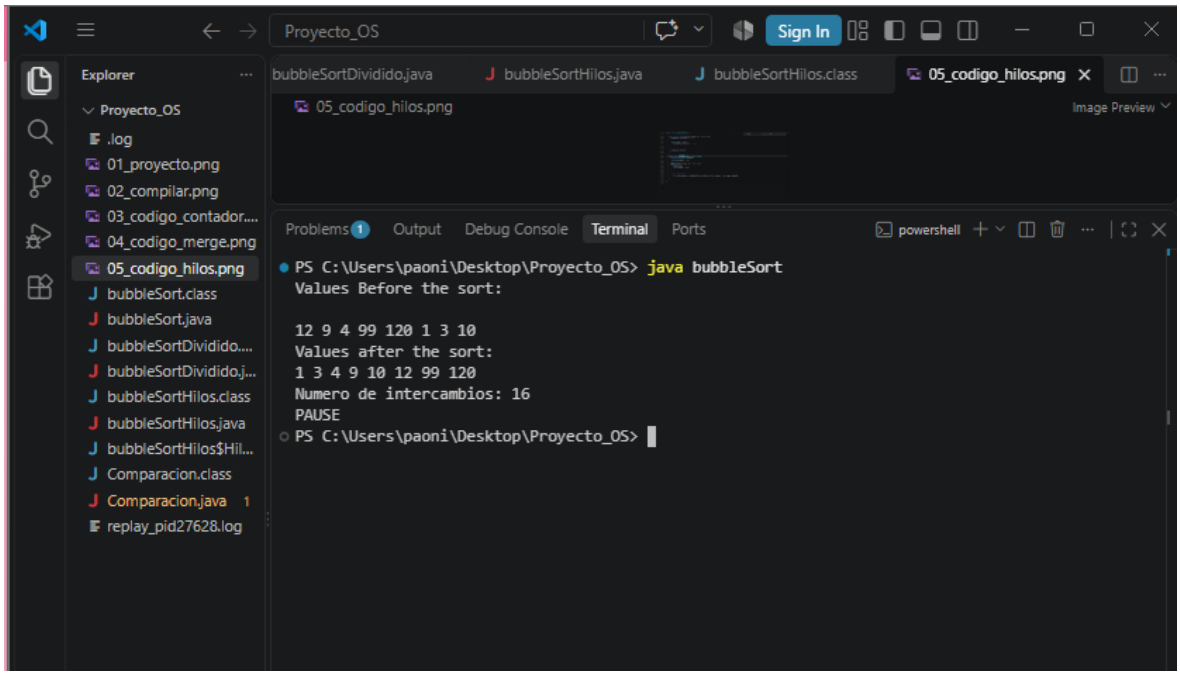


Figura 4. *Ejecución de java bubbleSort con el arreglo original {12, 9, 4, 99, 120, 1, 3, 10}.*

El arreglo de ocho elementos necesitó **16 intercambios**. Ese número coincide exactamente con la cantidad de inversiones del arreglo, es decir, con las parejas de números que están en el orden incorrecto, como el 12 que aparece antes que el 9. Cada intercambio del Bubble Sort corrige exactamente una inversión, por lo que el total de intercambios siempre es igual al número de inversiones (Cormen et al., 2022).

Pruebas con 10, 100, 1,000 y 10,000 elementos

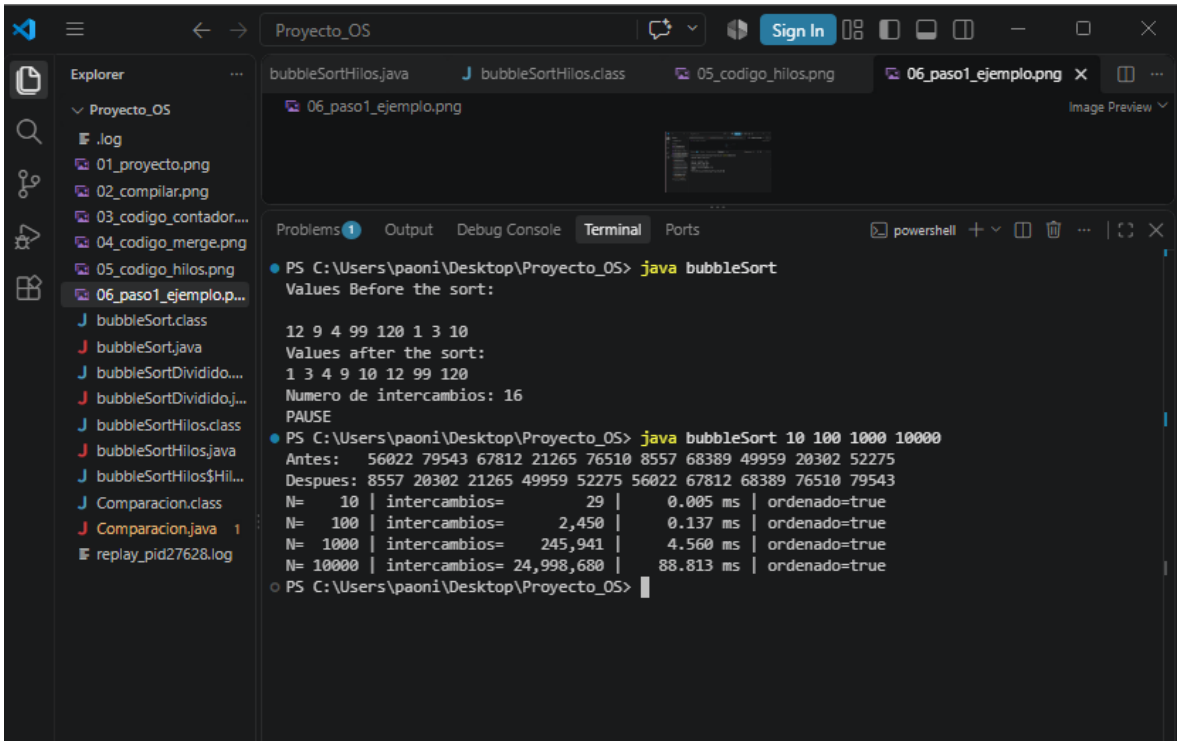


Figura 5. *Ejecución de java bubbleSort 10 100 1000 10000.*

En esta ejecución se obtuvieron 29, 2,450, 245,941 y 24,998,680 intercambios, con tiempos de 0.005 ms, 0.137 ms, 4.560 ms y 88.813 ms respectivamente. Todos los arreglos quedaron ordenados.

Resultados con puntos intermedios

Para la gráfica se agregaron puntos intermedios (50, 250, 500, 750, 2,500, 5,000 y 7,500 elementos) usando el programa `Comparacion`. La Tabla 2 compara los intercambios medidos con el número de inversiones del arreglo, calculado con otro algoritmo, y con el valor teórico esperado para datos aleatorios, n(n−1)/4.

Tabla 2. *Intercambios y tiempo del Paso 1 (tiempo como mediana de 5 ejecuciones).*

N	Intercambios medidos	Inversiones	Teórico n(n−1)/4	Medido / teórico	Tiempo (ms)
10	29	29	22	128.9 %	0.001
50	602	602	612	98.3 %	0.004
100	2,450	2,450	2,475	99.0 %	0.012
250	16,817	16,817	15,562	108.1 %	0.069
500	64,804	64,804	62,375	103.9 %	0.202
750	140,719	140,719	140,438	100.2 %	0.399
1,000	245,941	245,941	249,750	98.5 %	0.670

N	Intercambios medidos	Inversiones	Teórico n(n−1)/4	Medido / teórico	Tiempo (ms)
2,500	1,566,239	1,566,239	1,561,875	100.3 %	3.896
5,000	6,259,658	6,259,658	6,248,750	100.2 %	11.863
7,500	14,089,628	14,089,628	14,060,625	100.2 %	30.568
10,000	24,998,680	24,998,680	24,997,500	100.0 %	99.308

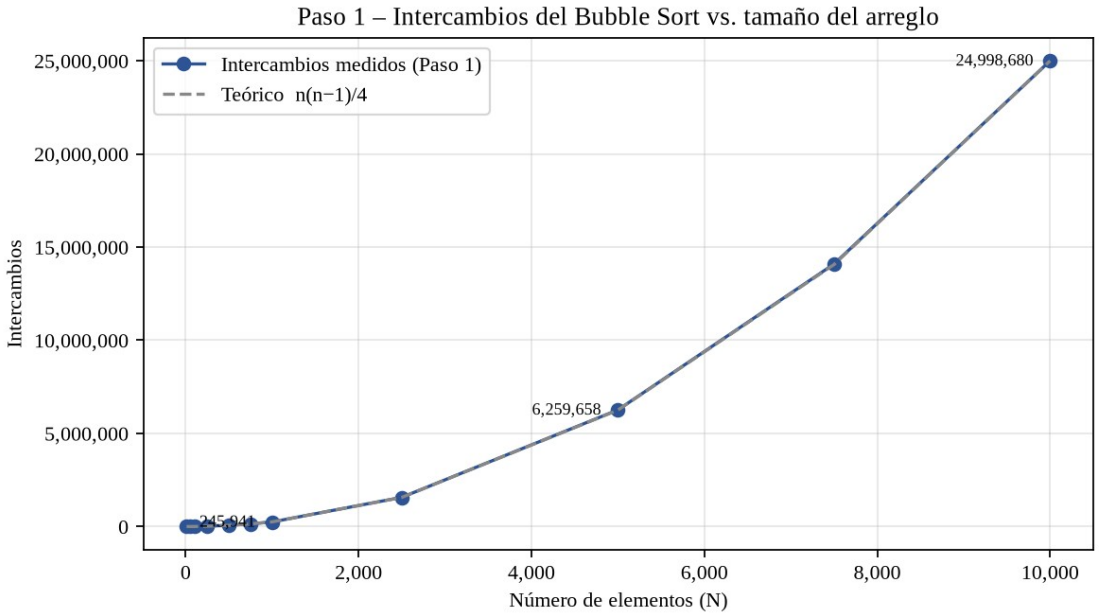


Figura 6. *Intercambios del Paso 1 según el número de elementos.*

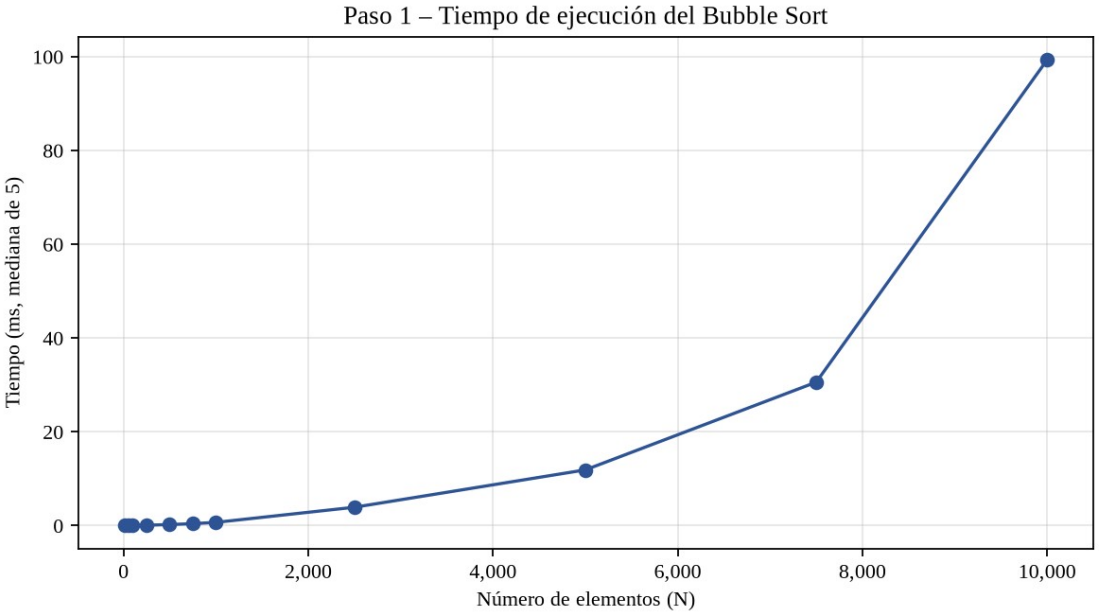


Figura 7. *Tiempo de ejecución del Paso 1 según el número de elementos.*

Interpretación de los resultados

La curva de intercambios es una parábola y no una recta. Al multiplicar N por 10, los intercambios se multiplican aproximadamente por 100: de 10 a 100 elementos pasan de 29 a 2,450, y de 1,000 a 10,000 pasan de 245,941 a 24,998,680. Esto confirma que el Bubble Sort es un algoritmo de orden $O(N^2)$.

El valor $n(n-1)/4$ se explica porque en un arreglo de N elementos existen $n(n-1)/2$ parejas posibles y, si los datos son aleatorios, cada pareja tiene la mitad de probabilidad de estar invertida. Como cada intercambio corrige una inversión, el promedio esperado de intercambios es $n(n-1)/4$. Con pocos elementos la diferencia puede ser grande (con 10 elementos se midieron 29 contra 22.5 esperados), pero a partir de 1,000 elementos la diferencia es menor al 2 %, porque con más datos el promedio se estabiliza. La columna de inversiones coincide exactamente con los intercambios en todos los casos.

Aunque el número de intercambios depende de los datos, el Bubble Sort siempre realiza $n(n-1)/2$ comparaciones, el doble del promedio de intercambios. Si el arreglo ya estuviera ordenado haría cero intercambios pero seguiría haciendo todas las comparaciones, y si estuviera en orden inverso haría $n(n-1)/2$ intercambios, que es el peor caso.

El tiempo también crece de forma cuadrática. Con 1,000 elementos tardó 0.670 ms y con 10,000 tardó 99.308 ms, unas 148 veces más con diez veces más datos; con 50,000 elementos ya tardó 2.89 segundos. El aumento es algo mayor que el factor 100 esperado porque un arreglo de 10,000 enteros ocupa unos 40 KB y ya no cabe en la caché L1 de datos de un núcleo (32 KB), así que el procesador tiene que ir más veces a memorias más lentas. Para N muy pequeños los tiempos son de microsegundos y están dominados por el ruido del sistema operativo y por el compilador JIT de Java; por eso cada medición se repitió cinco veces, se tomó la mediana y se hizo un calentamiento previo de la JVM.

Paso 2: División del arreglo en dos mitades y mezcla (merge)

Descripción del algoritmo

El programa `bubbleSortDividido.java` hace todo el trabajo en un solo hilo de ejecución. Primero divide el arreglo original en `left_array`, con la primera mitad, y `right_array`, con la segunda; si N es impar, la mitad derecha se queda con el elemento sobrante. Luego llama a `bubble_srt` para la mitad izquierda y, cuando termina, para la mitad derecha. Por último mezcla las dos listas ya ordenadas: compara el primer elemento pendiente de cada lista, copia el menor al resultado y avanza, hasta agotar ambas. El merge no hace intercambios, solo comparaciones y copias, y su costo es proporcional a N.

```
public static void ordenarYReportar(int[] var0, boolean var1) {
   int var2 = var0.length;
   if (var1) {
      imprimir("Antes:         ", var0);
   }

   long var3 = System.nanoTime();
   int var5 = var2 / 2;
   int[] var6 = new int[var5];
   int[] var7 = new int[var2 - var5];
   System.arraycopy(var0, 0, var6, 0, var5);
   System.arraycopy(var0, var5, var7, 0, var2 - var5);
   long var8 = bubble_srt(var6, var6.length);
   long var10 = bubble_srt(var7, var7.length);
   int[] var12 = merge(var6, var7);
   long var13 = System.nanoTime();
   if (var1) {
      imprimir("Mitad izquierda:", var6);
      imprimir("Mitad derecha:  ", var7);
      imprimir("Despues (merge):", var12);
   }

   System.out.printf("N=%6d | intercambios izq=%,10d der=%,10d total=%,11d | %9.3f ms | ordenado=%s%n", var2, var8, var10, var8 + var10, (double)(var13 - var3) / (double)10
}

public static long bubble_srt(int[] var0, int var1) {
```

Figura 8. *Método ordenarYReportar de bubbleSortDividido: división, ordenamiento de cada mitad y merge.*

Ejecución con el ejemplo del enunciado

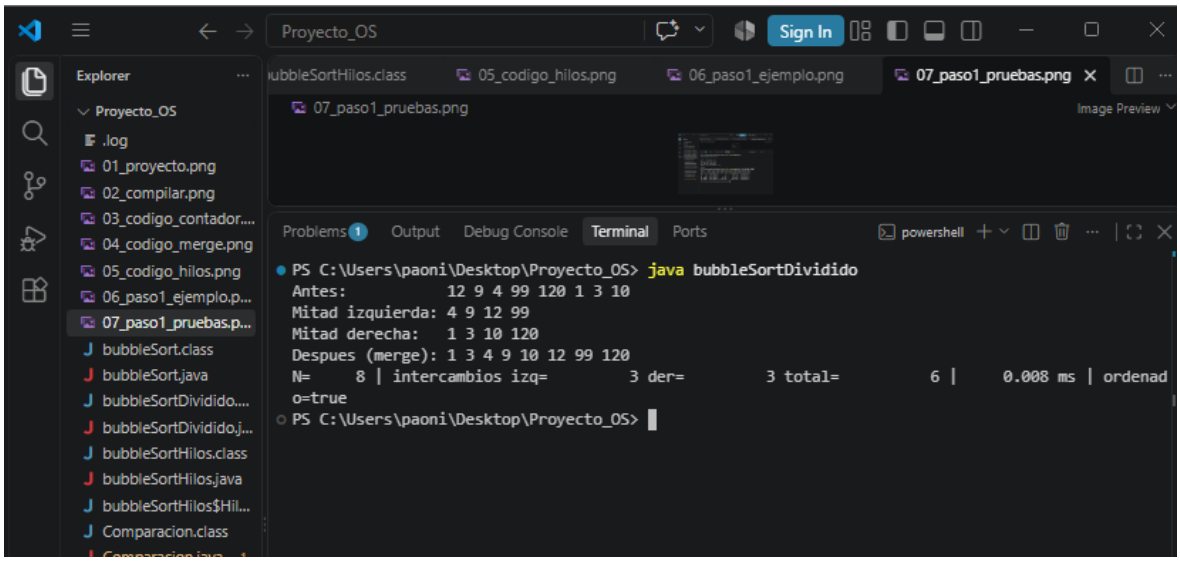


Figura 9. *Ejecución de java bubbleSortDividido con el arreglo de ocho números.*

Con el mismo arreglo de ocho números ahora solo se hicieron **6 intercambios**, tres en cada mitad, en lugar de 16. Las diez inversiones restantes eran entre un número de la mitad izquierda y uno de la derecha, como el 12 y el 1. Esas inversiones cruzadas las resuelve el merge sin hacer ningún intercambio.

Pruebas con 10, 100, 1,000 y 10,000 elementos

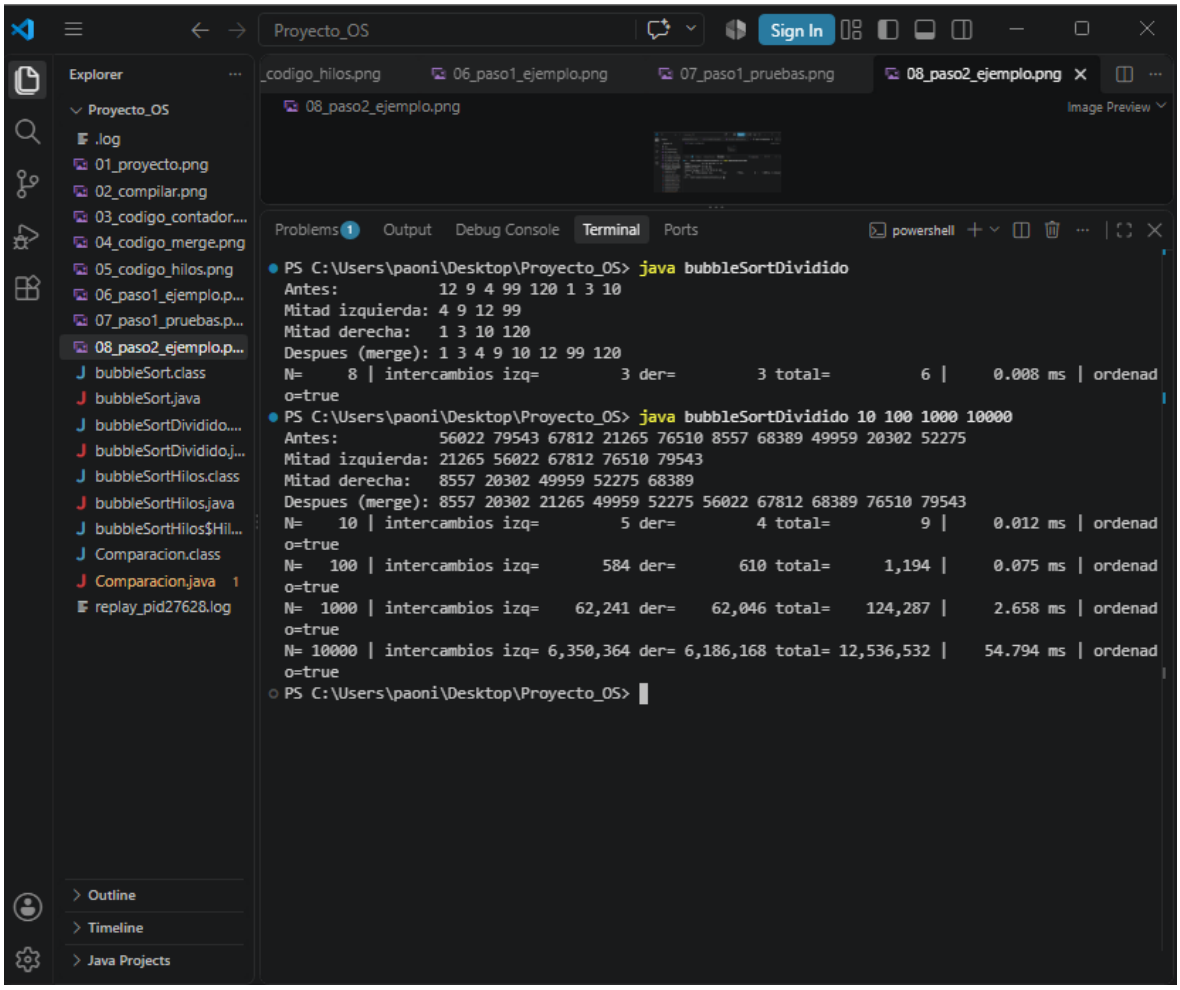


Figura 10. *Ejecución de java bubbleSortDividido 10 100 1000 10000.*

En esta ejecución los intercambios totales fueron 9, 1,194, 124,287 y 12,536,532, aproximadamente la mitad que en el Paso 1, con tiempos de 0.012 ms, 0.075 ms, 2.658 ms y 54.794 ms.

Comparación con el Paso 1

Tabla 3. *Paso 1 contra Paso 2 con los mismos datos. Una mejora mayor que 1 indica que el Paso 2 fue más rápido.*

N	Intercambios Paso 1	Intercambios Paso 2	Reducción	ms Paso 1	ms Paso 2	Mejora
10	29	9	69.0 %	0.001	0.005	0.13x
50	602	262	56.5 %	0.004	0.010	0.35x
100	2,450	1,194	51.3 %	0.012	0.016	0.74x
250	16,817	8,065	52.0 %	0.069	0.054	1.28x
500	64,804	30,051	53.6 %	0.202	0.143	1.41x

N	Intercambios Paso 1	Intercambios Paso 2	Reducción	ms Paso 1	ms Paso 2	Mejora
750	140,719	70,404	50.0 %	0.399	0.279	1.43x
1,000	245,941	124,287	49.5 %	0.670	0.489	1.37x
2,500	1,566,239	782,377	50.0 %	3.896	2.396	1.63x
5,000	6,259,658	3,092,817	50.6 %	11.863	6.492	1.83x
7,500	14,089,628	7,027,444	50.1 %	30.568	14.508	2.11x
10,000	24,998,680	12,536,532	49.9 %	99.308	26.461	3.75x
20,000	99,458,909	49,823,456	49.9 %	423.600	153.709	2.76x
50,000	622,516,767	311,317,434	50.0 %	2,886.907	1,381.363	2.09x

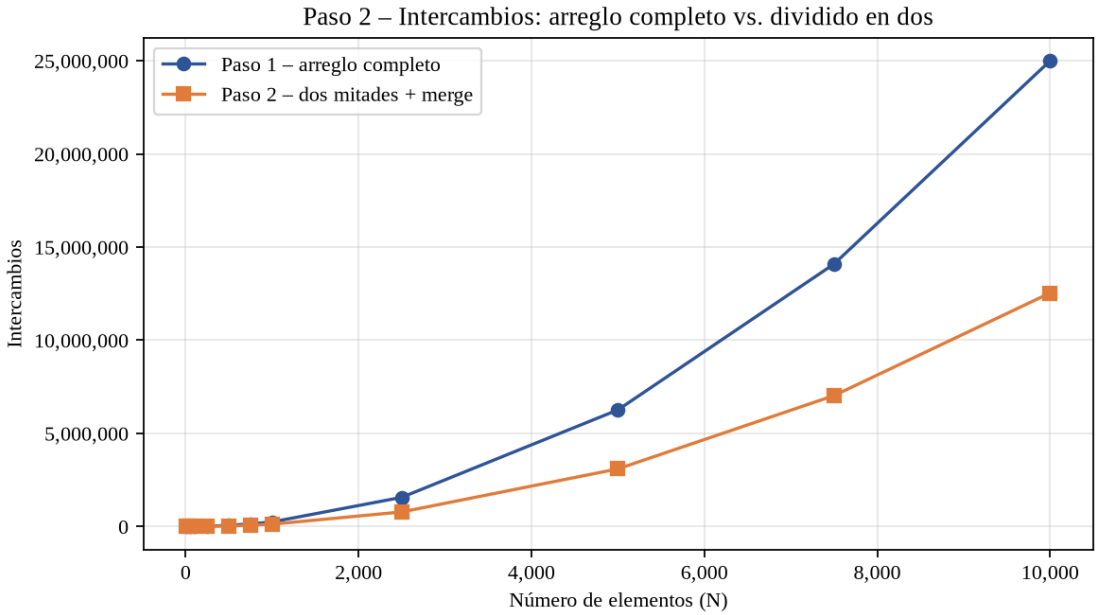


Figura 11. *Intercambios del arreglo completo contra el arreglo dividido en dos mitades.*

Conclusiones del Paso 2

El rendimiento sí mejoró, aunque el programa sigue siendo secuencial. En todos los tamaños los intercambios se redujeron aproximadamente a la mitad y, a partir de unos 250 elementos, también bajó el tiempo: con 10,000 elementos pasó de 99.308 ms a 26.461 ms y con 50,000 de 2.89 s a 1.38 s.

El factor principal es que el costo del Bubble Sort es cuadrático. Ordenar N elementos cuesta un tiempo proporcional a N^2, mientras que ordenar dos mitades cuesta $2 \cdot (N/2)^2 = N^2/2$, es decir, la mitad del trabajo, tanto en comparaciones como en intercambios. A esto se suma que aproximadamente la mitad de las inversiones son cruzadas, entre un elemento de la izquierda y uno de la derecha; el Bubble Sort las corregiría una por una, mientras que el merge las

resuelve con solo N comparaciones. El merge, además, es barato: su costo lineal es despreciable frente al costo cuadrático del ordenamiento cuando N es grande.

También influye la memoria caché. Cada mitad ocupa la mitad de la memoria y cabe mejor en la caché del procesador; con 10,000 elementos cada mitad ocupa unos 20 KB y sí entra en la caché L1, por eso la mejora medida en ese tamaño fue de 3.75x, mayor que el 2x teórico. Para tamaños muy grandes la mejora se estabiliza cerca de 2x (2.09x con 50,000 elementos).

Para N pequeños, entre 10 y 100 elementos, el Paso 2 fue igual o más lento, porque crear los dos arreglos, copiar los datos y hacer el merge cuesta más que el poco trabajo que se ahorra. Esta misma idea aplicada de forma recursiva, dividiendo cada mitad una y otra vez, es el algoritmo Merge Sort, que reduce la complejidad a O(N log N) (Cormen et al., 2022).

Paso 3: Ordenamiento de las dos mitades en hilos separados

Conceptos de hilos en Java

Un hilo (thread) es una secuencia de instrucciones que el sistema operativo puede planificar de forma independiente. Todos los hilos de un proceso comparten la misma memoria, pero cada uno tiene su propia pila y su propio contador de programa (Tanenbaum y Bos, 2015). En Java cada objeto `Thread` corresponde a un hilo real del sistema operativo, así que, si hay varios núcleos libres, el planificador puede ejecutar dos hilos al mismo tiempo en núcleos diferentes (Oracle, s.f.-a).

Para crear un hilo se escribe una clase que hereda de `Thread` y se coloca en el método `run` el código que ejecutará. El método `start` le pide al sistema operativo crear el hilo y ejecutar `run` en paralelo, sin que el hilo principal espere. El método `join` bloquea al hilo que lo llama hasta que el otro hilo termina, por lo que funciona como punto de sincronización (Oracle, s.f.-b). Este esquema, en el que el hilo principal reparte el trabajo, lo pone a correr y luego espera a que todo termine, se conoce como fork–join.

Implementación

Se creó la clase interna `HiloOrdenador`, que recibe su mitad del arreglo y la ordena dentro de `run`. Cada hilo guarda sus intercambios en su propia variable, de modo que los dos hilos nunca escriben en la misma memoria y no hace falta sincronización. El hilo principal divide el arreglo, crea el hilo izquierdo y el derecho, llama a `start` en ambos, espera con `join` a que los dos terminen y finalmente hace un único merge.



Figura 12. *Clase HiloOrdenador, que hereda de Thread y ordena su mitad en el método run.*

Ejecución

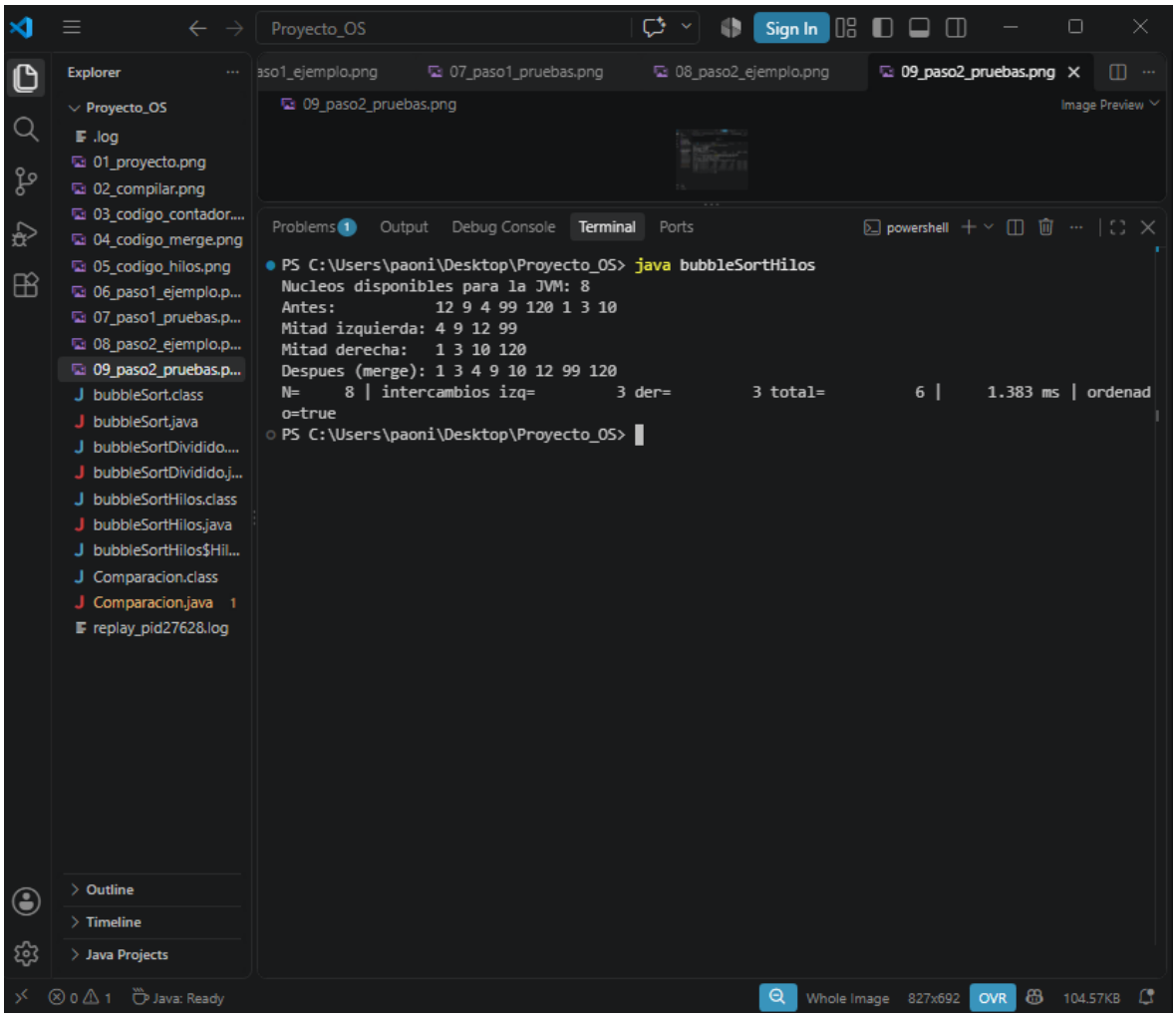


Figura 13. *Ejecución de java bubbleSortHilos con el ejemplo de ocho números.*

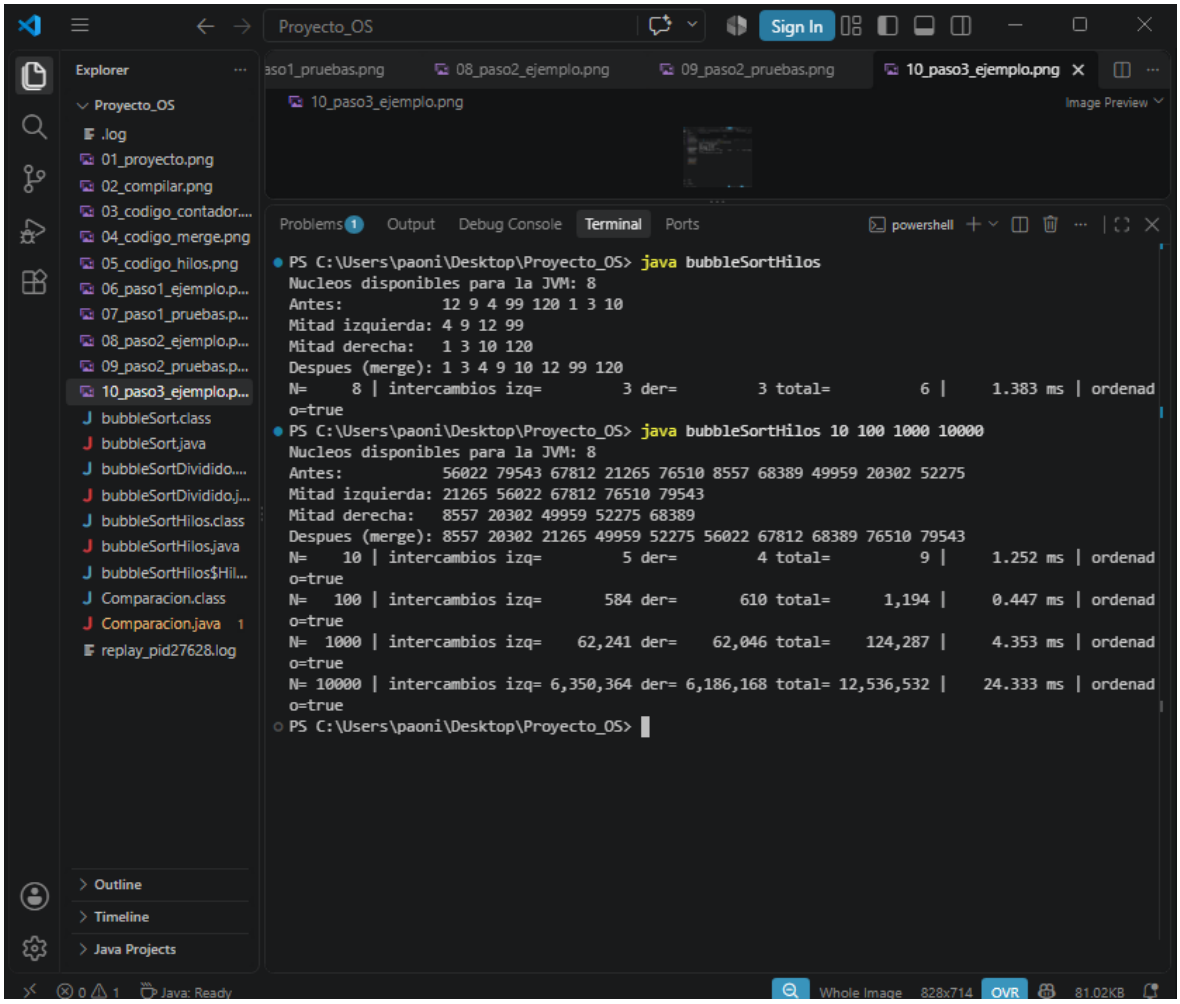


Figura 14. *Ejecución de java bubbleSortHilos 10 100 1000 10000.*

Los intercambios son idénticos a los del Paso 2, porque los hilos hacen exactamente el mismo trabajo, solo que al mismo tiempo. Lo que cambia es el tiempo: con 10,000 elementos la versión con hilos tardó 24.333 ms, menos de la mitad que el Paso 2 en la ejecución anterior (54.794 ms), pero con 10 elementos tardó 1.252 ms, muchísimo más que las versiones secuenciales. El programa también informa que la JVM ve 8 procesadores disponibles, que son los 8 procesadores lógicos del Ryzen 5 7520U.

Comparación de las tres versiones

Para comparar las tres versiones en igualdad de condiciones se ejecutó `java Comparacion`, que usa los mismos datos para los tres pasos, repite cada medición cinco veces y reporta la mediana. La Figura 15 muestra una de esas ejecuciones; los valores de las tablas

corresponden a la ejecución guardada en el archivo resultados.csv, por lo que los tiempos pueden diferir un poco de la captura, mientras que los intercambios son idénticos.

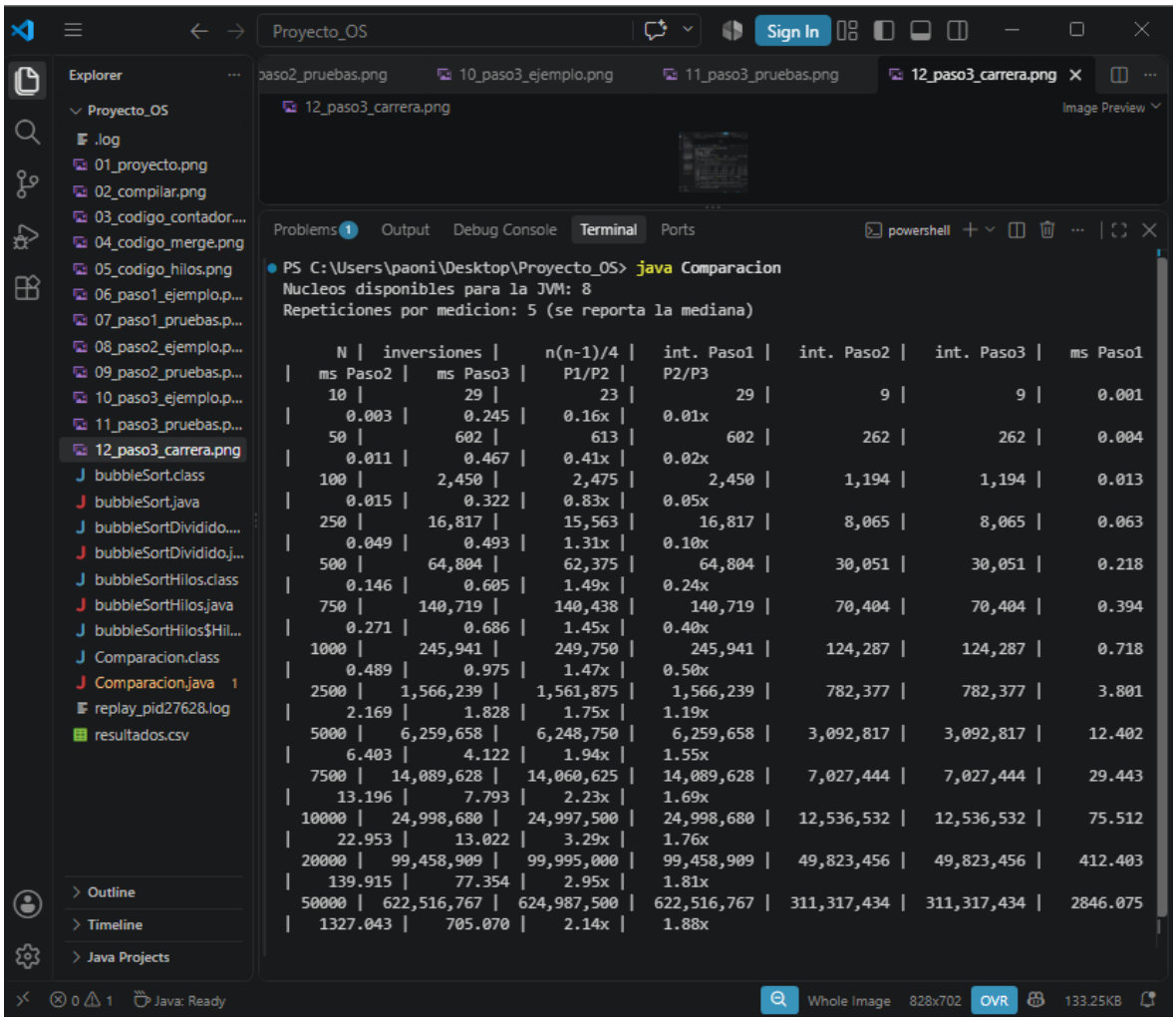


Figura 15. *Ejecución de java Comparacion con los tres pasos sobre los mismos datos.*

Tabla 4. *Tiempos de las tres versiones con los 8 procesadores lógicos (mediana de 5 ejecuciones). Los valores mayores que 1 en las dos últimas columnas indican que la versión con hilos fue más rápida.*

N	ms Paso 1	ms Paso 2	ms Paso 3	Paso 2 / Paso 3	Paso 1 / Paso 3
10	0.001	0.005	0.263	0.02x	0.00x
50	0.004	0.010	0.374	0.03x	0.01x
100	0.012	0.016	0.414	0.04x	0.03x
250	0.069	0.054	0.737	0.07x	0.09x
500	0.202	0.143	0.477	0.30x	0.42x
750	0.399	0.279	0.554	0.50x	0.72x
1,000	0.670	0.489	0.844	0.58x	0.79x
2,500	3.896	2.396	1.917	1.25x	2.03x
5,000	11.863	6.492	4.287	1.51x	2.77x
7,500	30.568	14.508	8.899	1.63x	3.44x
10,000	99.308	26.461	15.269	1.73x	6.50x

N	ms Paso 1	ms Paso 2	ms Paso 3	Paso 2 / Paso 3	Paso 1 / Paso 3
20,000	423.600	153.709	86.961	1.77x	4.87x
50,000	2,886.907	1,381.363	697.029	1.98x	4.14x

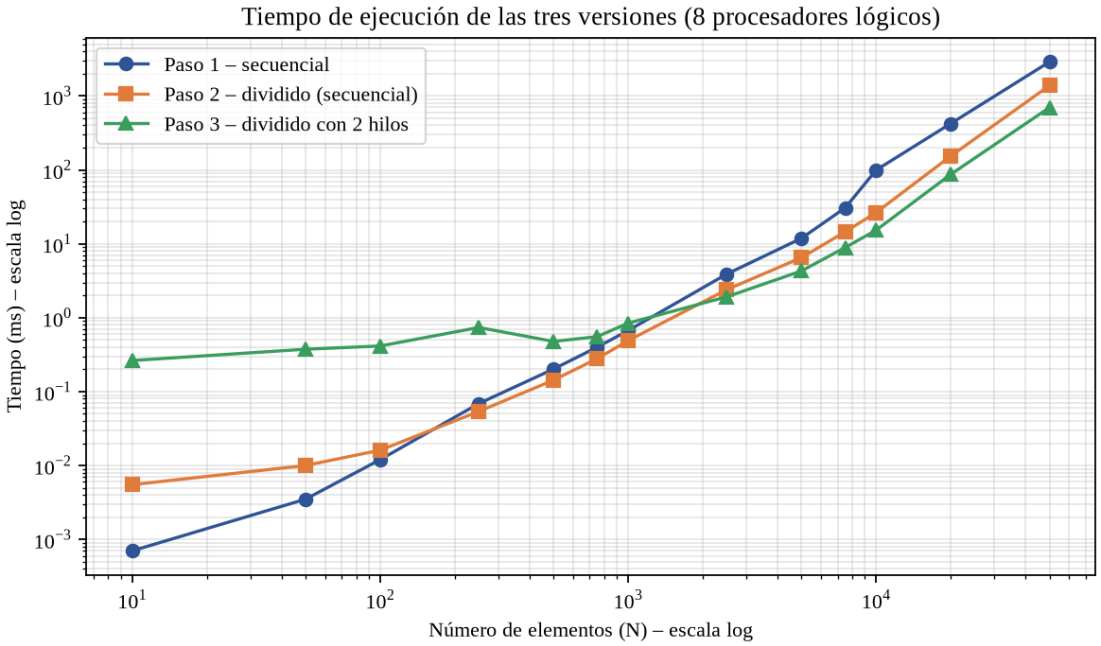


Figura 16. *Tiempo de las tres versiones en escala logarítmica.*

Uso de los núcleos durante la ejecución

Mientras se ejecutaban las pruebas se observó el Administrador de tareas de Windows. En la Figura 17, tomada durante la primera parte de `Comparacion`, el uso total del CPU se mantiene bajo, con picos cercanos al 15 %, porque la mayor parte del tiempo trabaja un solo hilo. En la Figura 18, al ejecutar `java bubbleSortHilos 100000`, el uso sube a valores entre 25 % y 35 %. Esto es coherente con lo esperado: el equipo tiene 8 procesadores lógicos, así que cada hilo que trabaja al 100 % representa aproximadamente un 12.5 % del total, y dos hilos trabajando a la vez representan alrededor del 25 %. En esa ejecución, ordenar 100,000 elementos con dos hilos tomó 3,392.665 ms y requirió 1,247,731,272 intercambios.

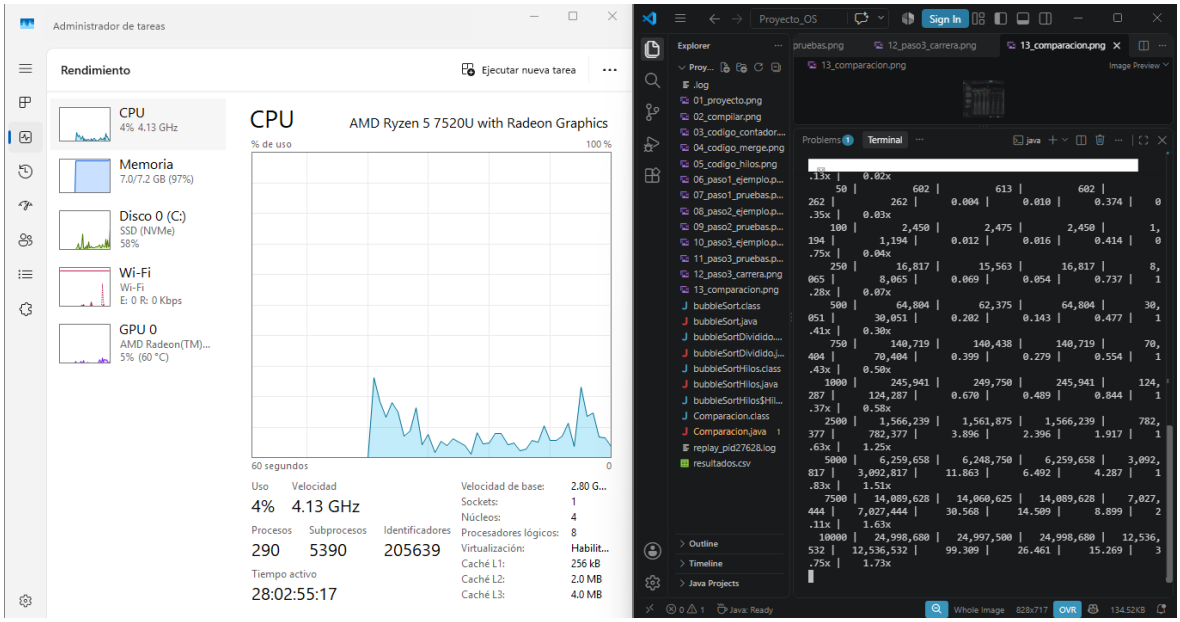


Figura 17. *Uso del CPU durante la primera parte de la ejecución de Comparacion.*

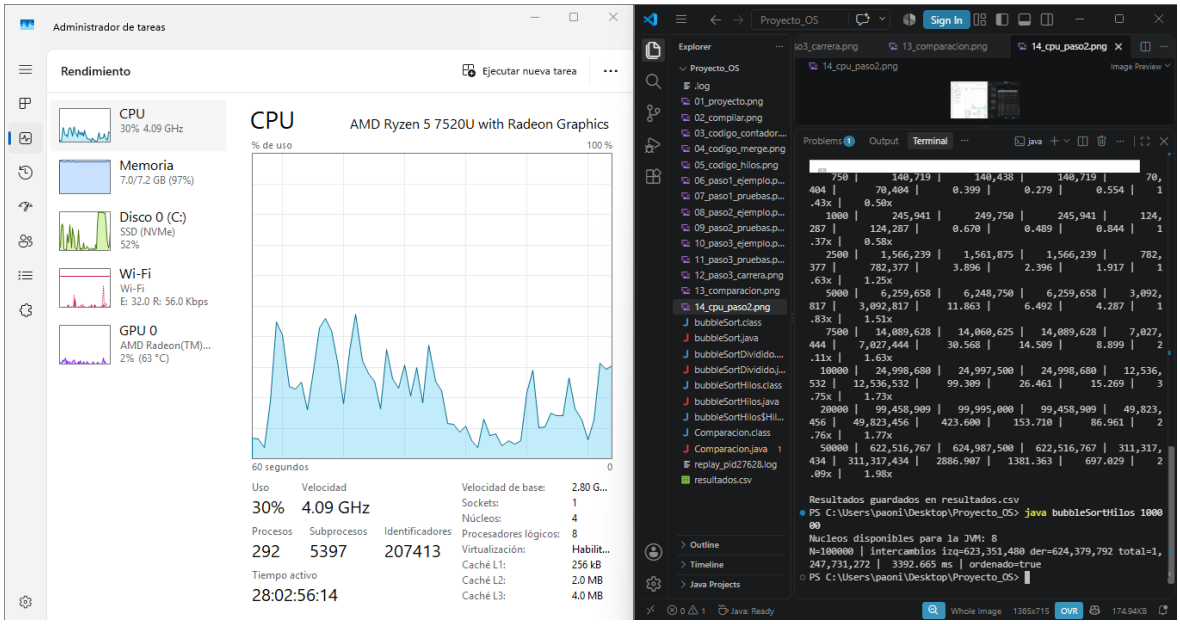


Figura 18. *Uso del CPU al ejecutar bubbleSortHilos con 100,000 elementos.*

La misma figura muestra que la memoria RAM estaba al 97 % de uso (7.0 de 7.2 GB), un dato que se relaciona con una de las dificultades descritas más adelante.

Simulación de un CPU single-core y dual-core

Para responder cómo operaría la versión con hilos en un CPU de uno o dos núcleos se repitió el experimento limitando los procesadores lógicos en los que Windows puede ejecutar el

programa, lo que se conoce como afinidad de CPU. Esto se hace con el comando `start /affinity` de CMD (Microsoft, 2023), donde el valor 1 permite usar solo el procesador lógico 0 y el valor 3 permite usar los procesadores lógicos 0 y 1. Como la terminal de Visual Studio Code es PowerShell, el comando se ejecutó a través de CMD con `cmd /c "start /affinity 1 /wait /b java Comparacion r_1nucleo.csv 5"` y su equivalente con afinidad 3.

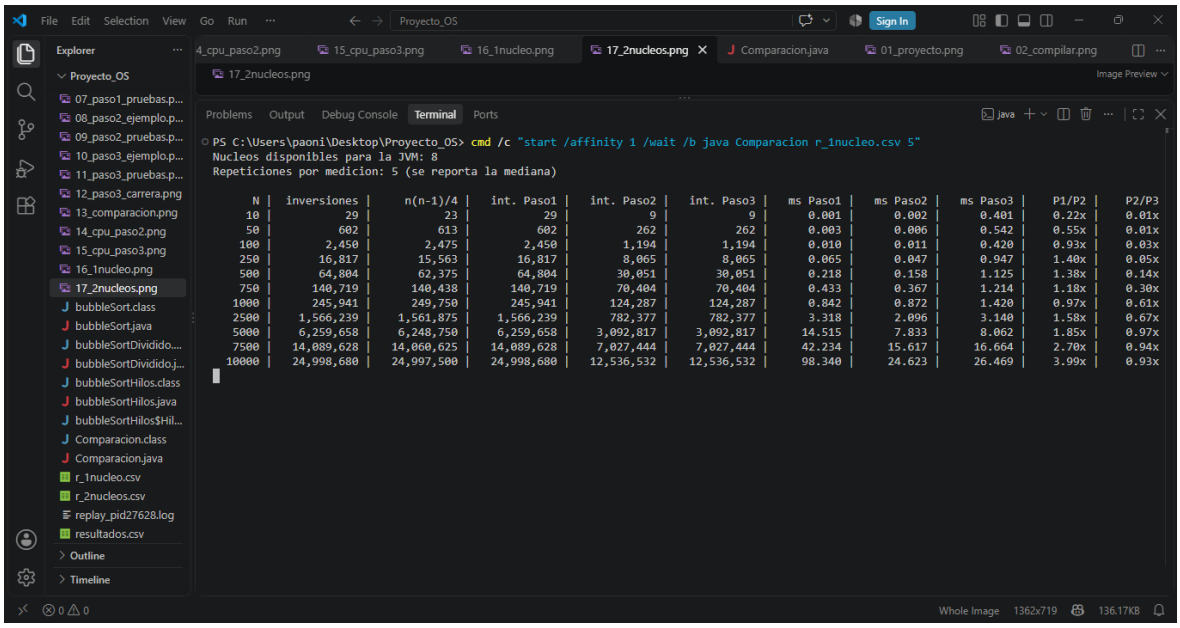


Figura 19. *Ejecución de Comparacion limitada a un procesador lógico (afinidad 1).*

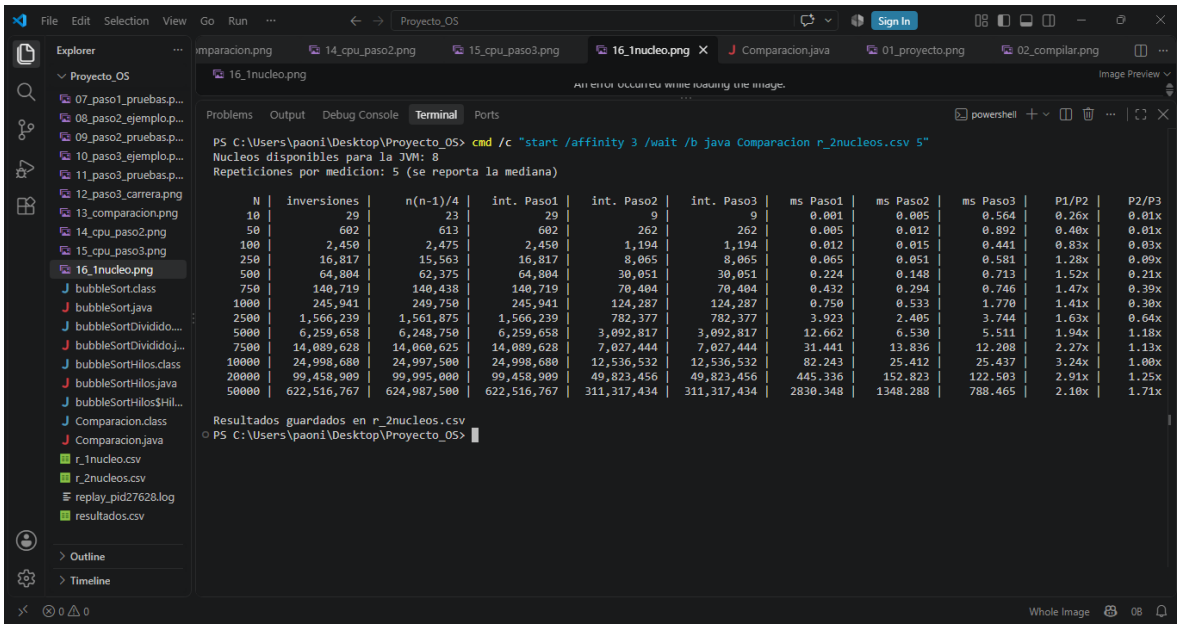


Figura 20. *Ejecución de Comparacion limitada a dos procesadores lógicos (afinidad 3).*

Las capturas muestran ejecuciones individuales, por lo que sus tiempos pueden variar un poco respecto a los de la Tabla 5, que provienen de los archivos r_1nucleo.csv y r_2nucleos.csv. En ambas ejecuciones el programa sigue indicando que la JVM ve 8 procesadores, porque en Windows el método `availableProcessors` reporta los procesadores del equipo y no la afinidad asignada. Sin embargo, los tiempos confirman que la restricción sí se aplicó: con un solo procesador lógico la versión con hilos dejó de ganar tiempo, como se ve en la Tabla 5 y en la Figura 21.

Tabla 5. *Paso 2 (secuencial) contra Paso 3 (hilos) con 1, 2 y 8 procesadores lógicos. Speedup = tiempo Paso 2 / tiempo Paso 3.*

N	1 proc. ms P2	1 proc. ms P3	1 proc. speedup	2 proc. ms P2	2 proc. ms P3	2 proc. speedup	8 proc. speedup
10	0.003	0.369	0.01x	0.005	0.564	0.01x	0.02x
50	0.006	0.499	0.01x	0.011	0.892	0.01x	0.03x
100	0.010	0.392	0.03x	0.015	0.441	0.03x	0.04x
250	0.044	0.537	0.08x	0.051	0.581	0.09x	0.07x
500	0.146	0.767	0.19x	0.148	0.713	0.21x	0.30x
750	0.281	0.957	0.29x	0.294	0.746	0.39x	0.50x
1,000	0.444	0.971	0.46x	0.533	1.770	0.30x	0.58x
2,500	2.010	2.687	0.75x	2.405	3.744	0.64x	1.25x
5,000	6.696	8.086	0.83x	6.530	5.511	1.19x	1.51x
7,500	14.459	16.183	0.89x	13.836	12.208	1.13x	1.63x
10,000	24.712	26.147	0.94x	25.412	25.437	1.00x	1.73x
20,000	171.356	165.735	1.03x	152.822	122.502	1.25x	1.77x
50,000	1,410.070	1,362.105	1.03x	1,348.288	788.465	1.71x	1.98x

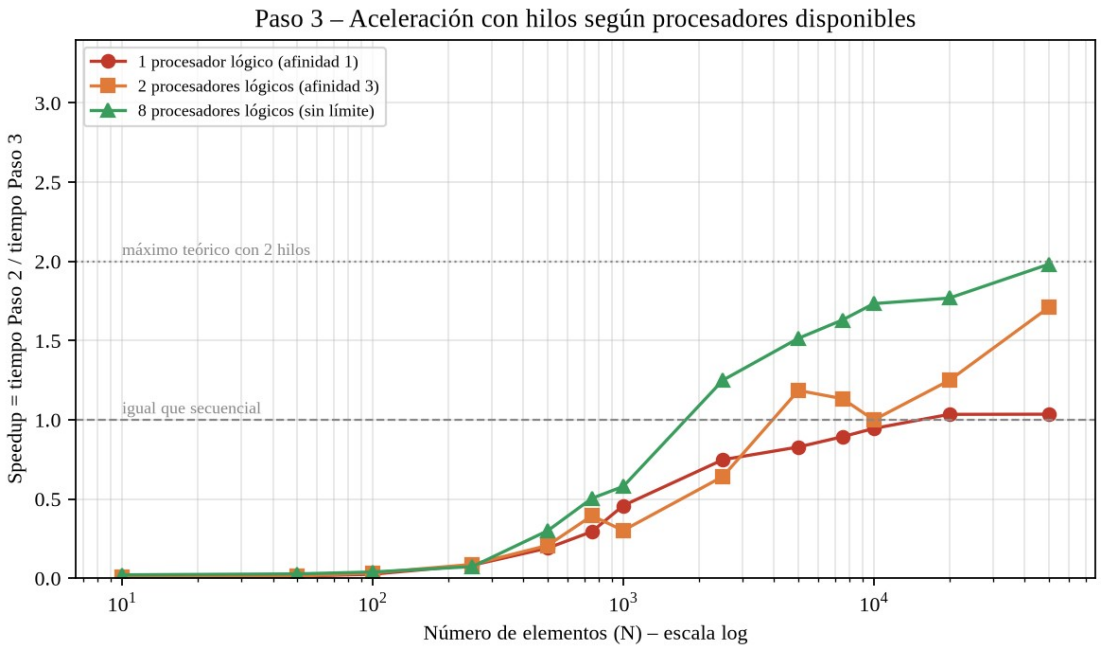


Figura 21. *Aceleración de la versión con hilos según los procesadores disponibles. Por encima de 1 los hilos ayudan y por debajo estorban.*

Conclusiones del Paso 3

Con la evidencia del Paso 2 se puede concluir que ejecutar las mitades en hilos es más eficiente, igual o peor según el tamaño del problema y los núcleos disponibles. Con todos los procesadores disponibles y N a partir de unos 2,500 elementos, los hilos fueron más rápidos. Con 50,000 elementos el Paso 3 tardó 697 ms contra 1381 ms del Paso 2 (1.98x) y contra 2887 ms del Paso 1 (4.14x). Es decir, dividir el problema aportó una mejora cercana a 2x y paralelizarlo aportó otra mejora cercana a 2x, muy próxima al máximo teórico para dos hilos.

Con N pequeño la versión con hilos fue peor. Crear e iniciar un hilo del sistema operativo cuesta del orden de décimas de milisegundo; con 10 elementos el Paso 3 tardó 0.263 ms, mientras que el Paso 2 tardó 0.005 ms. El paralelismo solo conviene cuando cada hilo tiene suficiente trabajo para compensar el costo de crearlo.

En un CPU de un solo núcleo la versión con hilos no mejora. Los dos hilos no pueden correr al mismo tiempo, así que el planificador los alterna por turnos (time-slicing) con cambios de contexto (Silberschatz et al., 2018). El trabajo total es el mismo que en el Paso 2, más el costo de crear los hilos y de los cambios de contexto; por eso el speedup medido con afinidad 1 quedó entre 0.83x y 1.03x para N de 5,000 en adelante, prácticamente igual que la versión

secuencial. Aun así, el Paso 3 en un núcleo sigue siendo más rápido que el Paso 1, porque conserva la ventaja algorítmica de dividir el arreglo.

En un CPU de dos núcleos reales cada hilo tendría su propio núcleo y la mejora podría acercarse a 2x, como se observó al usar todos los procesadores. La prueba con afinidad 3 dio una mejora menor, de 1.71x con 50,000 elementos y alrededor de 1x con 10,000. La razón es que en este procesador los procesadores lógicos 0 y 1 son los dos hilos de hardware (SMT) del mismo núcleo físico; ambos hilos de Java corrieron en un único núcleo que comparte sus unidades de ejecución y su caché, así que el resultado se parece más a un single-core con SMT que a un dual-core real. La mejora nunca llega exactamente a 2x porque dividir el arreglo, crear los hilos y hacer el merge son partes secuenciales, tal como describe la Ley de Amdahl (Amdahl, 1967), y porque las mitades pueden tener distinta cantidad de inversiones, de modo que un hilo termina antes y el otro determina el tiempo total.

Tener más de dos núcleos no ayuda más, porque el programa solo crea dos hilos y los demás núcleos quedan sin trabajo. Para aprovecharlos habría que dividir el arreglo en más partes, por ejemplo en tantas como procesadores reporte `availableProcessors`.

Demostración de una condición de carrera

Durante el desarrollo se probó primero usar un único contador estático compartido por ambos hilos, y el resultado era incorrecto y cambiaba en cada ejecución. Para demostrarlo, el programa acepta la opción `--compartido`, que además del contador propio de cada hilo incrementa un contador global sin sincronización.

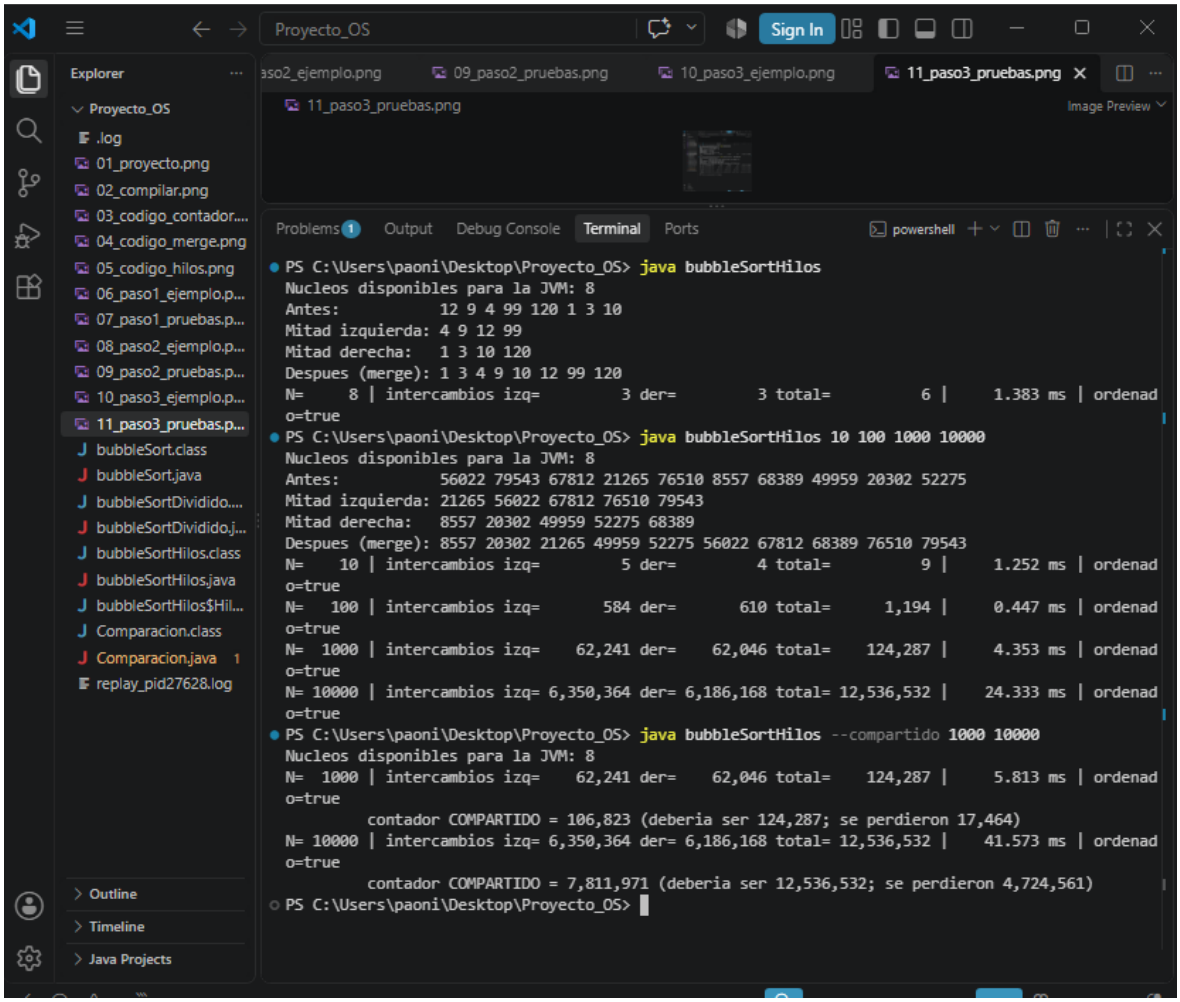


Figura 22. *Ejecución de java bubbleSortHilos --compartido 1000 10000.*

Con 1,000 elementos el contador compartido marcó 106,823 cuando debía marcar 124,287, y con 10,000 elementos marcó 7,811,971 cuando debía marcar 12,536,532, es decir, se perdieron 4,724,561 incrementos. La instrucción `contadorCompartido++` parece una sola operación, pero en realidad son tres: leer el valor, sumarle uno y escribirlo. Si los dos hilos leen el mismo valor al mismo tiempo, ambos escriben el mismo resultado y se pierde un incremento. Esto es una condición de carrera sobre una sección crítica (Silberschatz et al., 2018). Se puede resolver con `synchronized`, con `AtomicLong` o, como se hizo en la versión final, evitando compartir el dato: cada hilo cuenta en su propia variable y el hilo principal suma los dos resultados después del `join`, lo cual es correcto y no pierde rendimiento (Goetz et al., 2006).

Comentarios de implementación y dificultades encontradas

La primera dificultad apareció al copiar el código del enunciado desde el PDF. El texto "Values Before the sort:\n" quedó partido en dos líneas, lo cual no es válido para un String en Java, y la instrucción `j++` aparecía separada en dos renglones. Hubo que reconstruir esas líneas para que el programa compilara.

En el Paso 2 el enunciado sugiere llamar a `bubble_srt` con N/2 elementos para cada mitad, pero cuando N es impar la división entera redondea hacia abajo y se pierde un elemento. Se resolvió dejando que la mitad derecha tenga tamaño N − N/2. En el Paso 3 surgió la condición de carrera del contador compartido descrita en la sección anterior, que se resolvió con un contador por hilo. También se tuvo cuidado de llamar a `start` y no a `run`, porque `run` ejecuta el código en el hilo principal de forma secuencial y no produce paralelismo, y de no olvidar `join`, porque sin él el merge se haría mientras los hilos todavía están ordenando. Por eso cada programa verifica al final que el arreglo quedó ordenado.

Las primeras mediciones de tiempo variaban mucho entre ejecuciones porque Java compila el código mientras se ejecuta (compilador JIT) y el sistema operativo atiende otros procesos al mismo tiempo. Se resolvió usando `System.nanoTime`, una fase de calentamiento, cinco repeticiones con mediana y la misma semilla aleatoria para las tres versiones. Aun así, las Figuras 5, 10 y 14 muestran tiempos distintos a los de las tablas, porque son ejecuciones individuales sin calentamiento.

Al calcular el valor teórico n(n−1)/4 con el tipo `int`, el resultado para 50,000 elementos salió negativo, porque 50,000 × 49,999 es cerca de 2,500 millones y no cabe en un `int`. Se corrigió haciendo la operación con `long`, y por la misma razón todos los contadores de intercambios son `long`.

En cuanto al entorno, al principio el comando `javac` no era reconocido en la terminal, a pesar de que el JDK 27 estaba instalado, porque su carpeta bin no estaba en la variable PATH; se agregó la ruta del JDK a PATH. Después, el comando `start /affinity` falló en PowerShell, porque ahí `start` es un alias de otro comando (Start-Process) con otra sintaxis; se resolvió ejecutándolo a través de CMD con `cmd /c`. Además, el servidor de lenguaje Java de la extensión de Visual Studio Code se cerró una vez por falta de memoria y generó los

archivos hs_err_pid27628.log y replay_pid27628.log. Esto coincide con el 97 % de uso de RAM observado en el Administrador de tareas y no afectó a los programas del proyecto, que se compilaron y ejecutaron desde la terminal.

Por último, al simular el single-core y el dual-core se observó que `availableProcessors` seguía reportando 8 procesadores aunque la afinidad estuviera limitada, por lo que la verificación se hizo con los tiempos medidos. También se comprobó que la afinidad 3 no equivale a dos núcleos físicos, sino a los dos hilos SMT de un mismo núcleo; para simular un dual-core real se podría usar una afinidad que tome procesadores lógicos de núcleos distintos.

Conclusiones generales

El Bubble Sort realiza en promedio $n(n-1)/4$ intercambios, uno por cada inversión del arreglo, y su tiempo crece de forma cuadrática: multiplicar los datos por 10 multiplica el trabajo por cerca de 100. Dividir el problema en dos mitades y mezclarlas reduce el trabajo a la mitad aun sin paralelismo, porque $2 \cdot (N/2)^2 = N^2/2$ y el merge resuelve las inversiones cruzadas en tiempo lineal; esta es la idea en la que se basa el Merge Sort.

Ejecutar las mitades en hilos separados solo mejora el tiempo si hay núcleos libres para ejecutarlos al mismo tiempo y si el trabajo de cada hilo es mucho mayor que el costo de crearlo. En este equipo, con todos los procesadores disponibles, se obtuvo casi el máximo teórico de 2x frente al Paso 2, y la versión final fue unas 4 veces más rápida que el Bubble Sort original con 50,000 elementos. Con un solo procesador lógico los hilos no aportaron ninguna mejora.

También quedó claro que los hilos comparten memoria, lo cual es cómodo pero peligroso: cualquier dato que dos hilos modifiquen a la vez necesita sincronización o, mejor aún, un diseño que evite compartirlo. Todo esto se conecta con la gestión de procesos del sistema operativo, ya que los hilos de Java son hilos del sistema operativo y es su planificador el que decide en qué núcleo corre cada uno.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En Proceedings of the April 18-20, 1967, Spring Joint Computer Conference (pp. 483–485). ACM. https://doi.org/10.1145/1465482.1465560

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press.

Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., y Lea, D. (2006). Java concurrency in practice. Addison-Wesley.

Microsoft. (2023). start. Microsoft Learn. https://learn.microsoft.com/es-es/windows-server/administration/windows-commands/start

Oracle. (s.f.-a). Lesson: Concurrency. The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/concurrency/

Oracle. (s.f.-b). Class Thread. Java SE API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html

Pineda, R. A. (2026). Proyecto #1: Primeros pasos en la programación concurrente [Enunciado de proyecto]. Universidad Tecnológica Centroamericana.

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). Operating system concepts (10.ª ed.). Wiley.

Tanenbaum, A. S., y Bos, H. (2015). Modern operating systems (4.ª ed.). Pearson.

Anexo: Código fuente

A continuación se incluye el código fuente completo de los cuatro programas, tal como se compilaron y ejecutaron en el equipo de pruebas. El código se presenta en fuente Courier New para conservar la alineación.

bubbleSort.java (Paso 1)

```
import java.util.Random;

public class bubbleSort {

    public static void main(String a[]) {
        if (a.length == 0) {
            ejemploOriginal();
        } else {
            for (String arg : a) {
                probarConTamano(Integer.parseInt(arg));
            }
        }
    }

    public static void ejemploOriginal() {
        int i;
        int array[] = {12, 9, 4, 99, 120, 1, 3, 10};

        System.out.println("Values Before the sort:\n");
        for (i = 0; i < array.length; i++)
            System.out.print(array[i] + " ");
        System.out.println();

        long intercambios = bubble_srt(array, array.length);

        System.out.print("Values after the sort:\n");
        for (i = 0; i < array.length; i++)
            System.out.print(array[i] + " ");
        System.out.println();
        System.out.println("Numero de intercambios: " + intercambios);
        System.out.println("PAUSE");
    }

    public static void probarConTamano(int n) {
        int[] array = generarArreglo(n);
        if (n <= 20) imprimir("Antes:   ", array);

        long inicio = System.nanoTime();
        long intercambios = bubble_srt(array, array.length);
        long fin = System.nanoTime();

        if (n <= 20) imprimir("Despues: ", array);
        System.out.printf("N=%6d | intercambios=%,11d | %9.3f ms | ordenado=%s%n",
                n, intercambios, (fin - inicio) / 1e6, estaOrdenado(array));
    }


    public static long bubble_srt(int a[], int n) {
        int i, j, t = 0;
        long intercambios = 0;
        for (i = 0; i < n; i++) {
            for (j = 1; j < (n - i); j++) {
                if (a[j - 1] > a[j]) {
                    /* Inicio del intercambio */
```

```
                    t = a[j - 1];
                    a[j - 1] = a[j];
                    a[j] = t;
                    /* Final del intercambio */
                    intercambios++;
                }
            }
        }
        return intercambios;
    }

    public static int[] generarArreglo(int n) {
        Random r = new Random(12345L + n);
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = r.nextInt(100000);
        return arr;
    }

    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++)
            if (arr[i - 1] > arr[i]) return false;
        return true;
    }

    public static void imprimir(String etiqueta, int[] arr) {
        System.out.print(etiqueta);
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    }
}
```

bubbleSortDividido.java (Paso 2)

```
import java.util.Random;

public class bubbleSortDividido {

    public static void main(String a[]) {
        if (a.length == 0) {
            int array[] = {12, 9, 4, 99, 120, 1, 3, 10};
            ordenarYReportar(array, true);
        } else {
            for (String arg : a) {
                int n = Integer.parseInt(arg);
                ordenarYReportar(generarArreglo(n), n <= 20);
            }
        }
    }

    public static void ordenarYReportar(int[] array, boolean mostrar) {
        int n = array.length;
        if (mostrar) imprimir("Antes:          ", array);

        long inicio = System.nanoTime();
        int mitad = n / 2;
        int left_array[] = new int[mitad];
        int right_array[] = new int[n - mitad];
        System.arraycopy(array, 0, left_array, 0, mitad);
        System.arraycopy(array, mitad, right_array, 0, n - mitad);

        long intIzq = bubble_srt(left_array, left_array.length);
        long intDer = bubble_srt(right_array, right_array.length);

        int[] resultado = merge(left_array, right_array);
```

```
        long fin = System.nanoTime();

        if (mostrar) {
            imprimir("Mitad izquierda:", left_array);
            imprimir("Mitad derecha:  ", right_array);
            imprimir("Despues (merge):", resultado);
        }
        System.out.printf("N=%6d | intercambios izq=%,10d der=%,10d total=%,11d |
%9.3f ms | ordenado=%s%n",
                n, intIzq, intDer, intIzq + intDer, (fin - inicio) / 1e6,
estaOrdenado(resultado));
    }

    public static long bubble_srt(int a[], int n) {
        int i, j, t = 0;
        long intercambios = 0;
        for (i = 0; i < n; i++) {
            for (j = 1; j < (n - i); j++) {
                if (a[j - 1] > a[j]) {
                    t = a[j - 1];
                    a[j - 1] = a[j];
                    a[j] = t;
                    intercambios++;
                }
            }
        }
        return intercambios;
    }


    public static int[] merge(int[] left, int[] right) {
        int[] res = new int[left.length + right.length];
        int i = 0, j = 0, k = 0;
        while (i < left.length && j < right.length) {
            if (left[i] <= right[j]) res[k++] = left[i++];
            else res[k++] = right[j++];
        }
        while (i < left.length) res[k++] = left[i++];
        while (j < right.length) res[k++] = right[j++];
        return res;
    }

    public static int[] generarArreglo(int n) {
        Random r = new Random(12345L + n); // misma semilla que el Paso 1 -> mismos
datos
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = r.nextInt(100000);
        return arr;
    }

    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++)
            if (arr[i - 1] > arr[i]) return false;
        return true;
    }

    public static void imprimir(String etiqueta, int[] arr) {
        System.out.print(etiqueta + " ");
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    }
}
```

bubbleSortHilos.java (Paso 3)

```
import java.util.Random;


public class bubbleSortHilos {

    static long contadorCompartido = 0;
    static boolean usarCompartido = false;


    static class HiloOrdenador extends Thread {
        private final int[] arreglo;
        long intercambios = 0;

        HiloOrdenador(String nombre, int[] arreglo) {
            super(nombre);
            this.arreglo = arreglo;
        }

        @Override
        public void run() {
            intercambios = bubble_srt(arreglo, arreglo.length);
        }
    }

    public static void main(String a[]) throws InterruptedException {
        int idx = 0;
        if (a.length > 0 && a[0].equals("--compartido")) {
            usarCompartido = true;
            idx = 1;
        }
        System.out.println("Nucleos disponibles para la JVM: " +
Runtime.getRuntime().availableProcessors());
        if (idx == a.length) {
            int array[] = {12, 9, 4, 99, 120, 1, 3, 10};
            ordenarYReportar(array, true);
        } else {
            for (int k = idx; k < a.length; k++) {
                int n = Integer.parseInt(a[k]);
                ordenarYReportar(generarArreglo(n), n <= 20);
            }
        }
    }

    public static void ordenarYReportar(int[] array, boolean mostrar) throws
InterruptedException {
        int n = array.length;
        contadorCompartido = 0;
        if (mostrar) imprimir("Antes:          ", array);

        long inicio = System.nanoTime();



        int mitad = n / 2;
        int left_array[] = new int[mitad];
        int right_array[] = new int[n - mitad];
        System.arraycopy(array, 0, left_array, 0, mitad);
        System.arraycopy(array, mitad, right_array, 0, n - mitad);

        HiloOrdenador hiloIzq = new HiloOrdenador("Hilo-Izquierdo", left_array);
        HiloOrdenador hiloDer = new HiloOrdenador("Hilo-Derecho", right_array);
        hiloIzq.start();
        hiloDer.start();
```

```
        hiloIzq.join();
        hiloDer.join();

        int[] resultado = merge(left_array, right_array);

        long fin = System.nanoTime();

        if (mostrar) {
            imprimir("Mitad izquierda:", left_array);
            imprimir("Mitad derecha:  ", right_array);
            imprimir("Despues (merge):", resultado);
        }
        long total = hiloIzq.intercambios + hiloDer.intercambios;
        System.out.printf("N=%6d | intercambios izq=%,10d der=%,10d total=%,11d |
%9.3f ms | ordenado=%s%n",
                n, hiloIzq.intercambios, hiloDer.intercambios, total, (fin - inicio)
/ 1e6, estaOrdenado(resultado));
        if (usarCompartido) {
            System.out.printf("         contador COMPARTIDO = %,d (deberia ser %,d;
se perdieron %,d)%n",
                    contadorCompartido, total, total - contadorCompartido);
        }
    }

    public static long bubble_srt(int a[], int n) {
        int i, j, t = 0;
        long intercambios = 0;
        for (i = 0; i < n; i++) {
            for (j = 1; j < (n - i); j++) {
                if (a[j - 1] > a[j]) {
                    t = a[j - 1];
                    a[j - 1] = a[j];
                    a[j] = t;
                    intercambios++;
                    if (usarCompartido) contadorCompartido++; // carrera: leer-
sumar-escribir no es atomico
                }
            }
        }
        return intercambios;
    }

    public static int[] merge(int[] left, int[] right) {
        int[] res = new int[left.length + right.length];
        int i = 0, j = 0, k = 0;
        while (i < left.length && j < right.length) {
            if (left[i] <= right[j]) res[k++] = left[i++];
            else res[k++] = right[j++];
        }
        while (i < left.length) res[k++] = left[i++];
        while (j < right.length) res[k++] = right[j++];
        return res;
    }

    public static int[] generarArreglo(int n) {
        Random r = new Random(12345L + n); // misma semilla que Paso 1 y Paso 2
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = r.nextInt(100000);
        return arr;
    }

    public static boolean estaOrdenado(int[] arr) {
        for (int i = 1; i < arr.length; i++)
            if (arr[i - 1] > arr[i]) return false;
        return true;
    }
```

```
    public static void imprimir(String etiqueta, int[] arr) {
        System.out.print(etiqueta + " ");
        for (int x : arr) System.out.print(x + " ");
        System.out.println();
    }
}
```

Comparacion.java (experimento comparativo)

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Arrays;


public class Comparacion {

    static final int[] TAMANOS_DEFECTO = {10, 50, 100, 250, 500, 750, 1000, 2500,
5000, 7500, 10000, 20000, 50000};

    public static void main(String[] args) throws Exception {
        String archivo = args.length > 0 ? args[0] : "resultados.csv";
        int repeticiones = args.length > 1 ? Integer.parseInt(args[1]) : 5;
        int[] tamanos = TAMANOS_DEFECTO;
        if (args.length > 2) {
            tamanos = new int[args.length - 2];
            for (int i = 2; i < args.length; i++) tamanos[i - 2] =
Integer.parseInt(args[i]);
        }

        System.out.println("Nucleos disponibles para la JVM: " +
Runtime.getRuntime().availableProcessors());
        System.out.println("Repeticiones por medicion: " + repeticiones + " (se
reporta la mediana)");
        calentarJVM();

        System.out.printf("%n%7s | %12s | %12s | %12s | %12s | %12s | %10s | %10s |
%10s | %8s | %8s%n",
                "N", "inversiones", "n(n-1)/4", "int. Paso1", "int. Paso2", "int.
Paso3",
                "ms Paso1", "ms Paso2", "ms Paso3", "P1/P2", "P2/P3");

        try (PrintWriter csv = new PrintWriter(new FileWriter(archivo))) {

csv.println("N,inversiones,teorico_n(n-1)/4,intercambios_paso1,intercambios_paso2,in
tercambios_paso3,"
                    +
"ms_paso1,ms_paso2,ms_paso3,speedup_p1_vs_p2,speedup_p2_vs_p3,speedup_p1_vs_p3");
            for (int n : tamanos) {
                int[] original = bubbleSort.generarArreglo(n);
                int[] esperado = original.clone();
                Arrays.sort(esperado);

                long inversiones = contarInversiones(original.clone());
                double teorico = n * (n - 1L) / 4.0;

                long[] r1 = new long[1], r2 = new long[1], r3 = new long[1];
                double[] t1 = new double[repeticiones], t2 = new
double[repeticiones], t3 = new double[repeticiones];
                for (int k = 0; k < repeticiones; k++) {
                    t1[k] = medirPaso1(original, esperado, r1);
                    t2[k] = medirPaso2(original, esperado, r2);
                    t3[k] = medirPaso3(original, esperado, r3);
                }
                double m1 = mediana(t1), m2 = mediana(t2), m3 = mediana(t3);
```

```
                System.out.printf("%7d | %,12d | %,12.0f | %,12d | %,12d | %,12d |
%10.3f | %10.3f | %10.3f | %7.2fx | %7.2fx%n",
                        n, inversiones, teorico, r1[0], r2[0], r3[0], m1, m2, m3, m1
/ m2, m2 / m3);
                csv.printf(java.util.Locale.US, "%d,%d,%.1f,%d,%d,%d,%.4f,%.4f,%.4f,
%.3f,%.3f,%.3f%n",
                        n, inversiones, teorico, r1[0], r2[0], r3[0], m1, m2, m3, m1
/ m2, m2 / m3, m1 / m3);
            }
        }
        System.out.println("\nResultados guardados en " + archivo);
    }

    static double medirPaso1(int[] original, int[] esperado, long[] intercambios) {
        int[] a = original.clone();
        long ini = System.nanoTime();
        intercambios[0] = bubbleSort.bubble_srt(a, a.length);
        long fin = System.nanoTime();
        verificar(a, esperado, "Paso 1");
        return (fin - ini) / 1e6;
    }

    static double medirPaso2(int[] original, int[] esperado, long[] intercambios) {
        int[] a = original.clone();
        long ini = System.nanoTime();
        int mitad = a.length / 2;
        int[] left = Arrays.copyOfRange(a, 0, mitad);
        int[] right = Arrays.copyOfRange(a, mitad, a.length);
        long i1 = bubbleSortDividido.bubble_srt(left, left.length);
        long i2 = bubbleSortDividido.bubble_srt(right, right.length);
        int[] res = bubbleSortDividido.merge(left, right);
        long fin = System.nanoTime();
        intercambios[0] = i1 + i2;
        verificar(res, esperado, "Paso 2");
        return (fin - ini) / 1e6;
    }

    static double medirPaso3(int[] original, int[] esperado, long[] intercambios) {
        int[] a = original.clone();
        long ini = System.nanoTime();
        int mitad = a.length / 2;
        int[] left = Arrays.copyOfRange(a, 0, mitad);
        int[] right = Arrays.copyOfRange(a, mitad, a.length);
        bubbleSortHilos.HiloOrdenador h1 = new bubbleSortHilos.HiloOrdenador("Hilo-
Izquierdo", left);
        bubbleSortHilos.HiloOrdenador h2 = new bubbleSortHilos.HiloOrdenador("Hilo-
Derecho", right);
        h1.start();
        h2.start();
        try {
            h1.join();
            h2.join();
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        int[] res = bubbleSortHilos.merge(left, right);
        long fin = System.nanoTime();
        intercambios[0] = h1.intercambios + h2.intercambios;
        verificar(res, esperado, "Paso 3");
        return (fin - ini) / 1e6;
    }

    static void calentarJVM() {
        int[] datos = bubbleSort.generarArreglo(3000);
        int[] esp = datos.clone();
        Arrays.sort(esp);
```

```
        long[] dummy = new long[1];
        for (int i = 0; i < 10; i++) {
            medirPaso1(datos, esp, dummy);
            medirPaso2(datos, esp, dummy);
            medirPaso3(datos, esp, dummy);
        }
    }

    static void verificar(int[] obtenido, int[] esperado, String paso) {
        if (!Arrays.equals(obtenido, esperado))
            throw new IllegalStateException(paso + " no ordeno correctamente");
    }

    static double mediana(double[] v) {
        double[] c = v.clone();
        Arrays.sort(c);
        int m = c.length / 2;
        return c.length % 2 == 1 ? c[m] : (c[m - 1] + c[m]) / 2.0;
    }


    static long contarInversiones(int[] a) {
        if (a.length < 2) return 0;
        int mitad = a.length / 2;
        int[] izq = Arrays.copyOfRange(a, 0, mitad);
        int[] der = Arrays.copyOfRange(a, mitad, a.length);
        long inv = contarInversiones(izq) + contarInversiones(der);
        int i = 0, j = 0, k = 0;
        while (i < izq.length && j < der.length) {
            if (izq[i] <= der[j]) a[k++] = izq[i++];
            else {
                a[k++] = der[j++];
                inv += izq.length - i;
            }
        }
        while (i < izq.length) a[k++] = izq[i++];
        while (j < der.length) a[k++] = der[j++];
        return inv;
    }
}
```